Bank Account *Management System :*

Problem Statement: Design a system to manage bank accounts.

Steps :

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account),

overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like

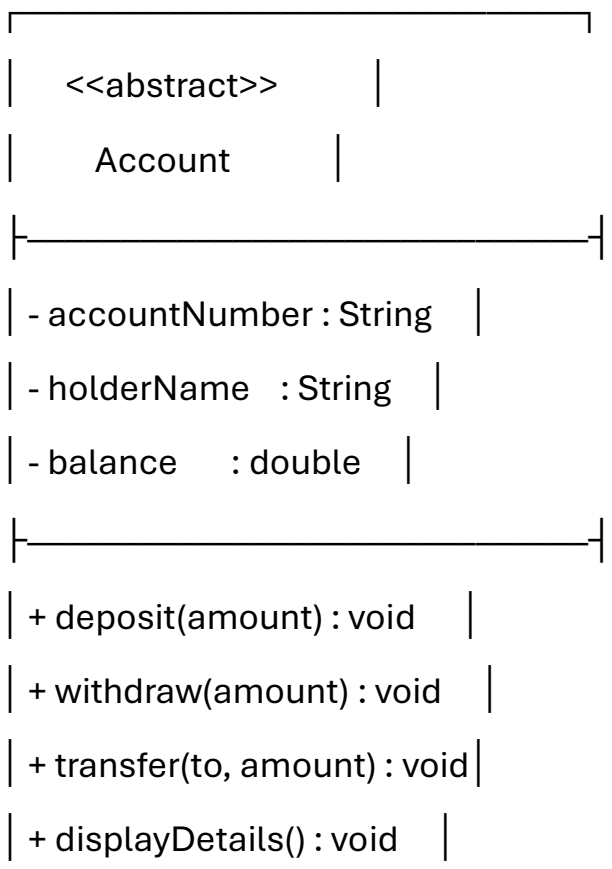
deposits, withdrawals, and transfers.

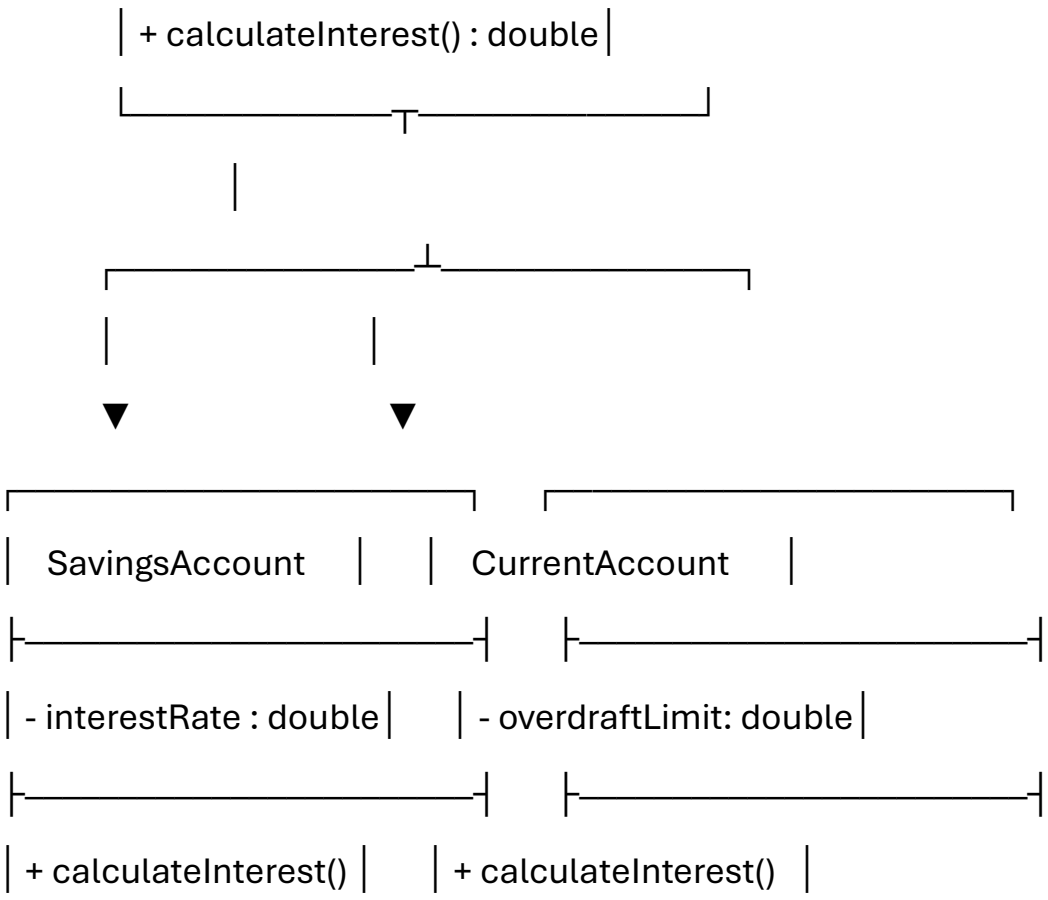
Algorithm :

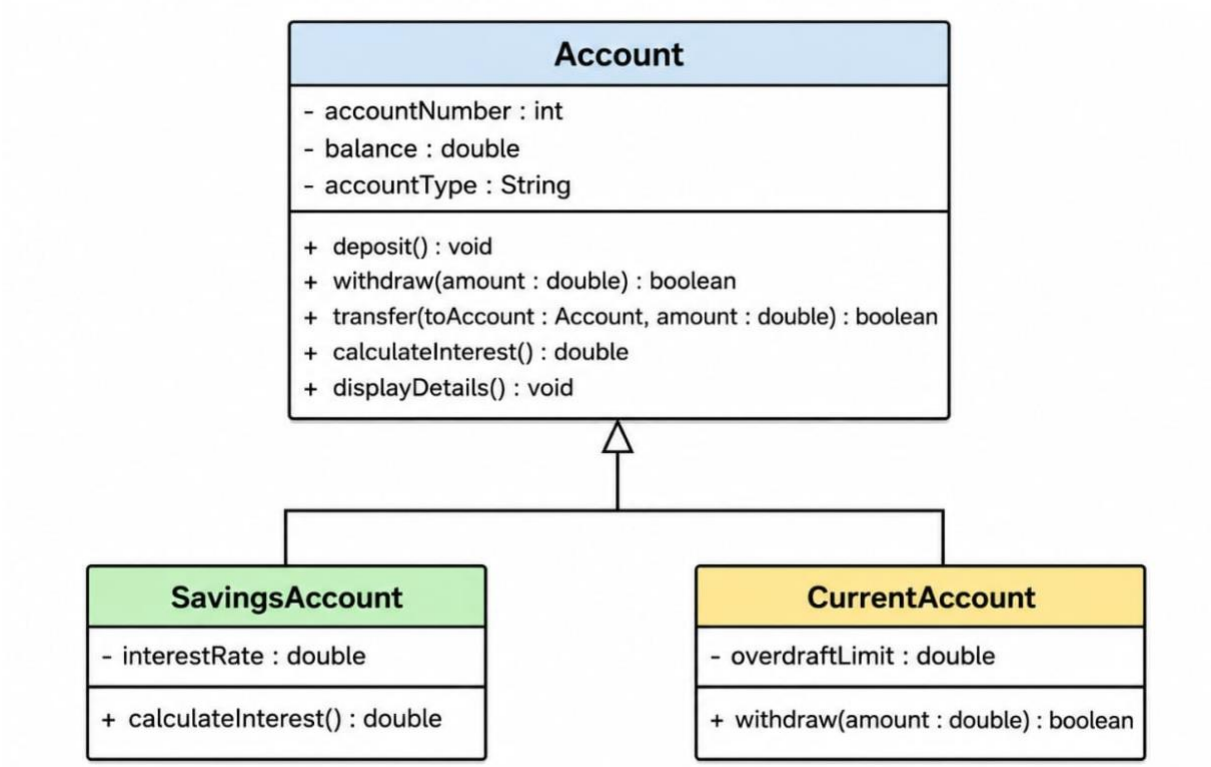
- Create an abstract class named Account containing common attributes such as account number, account holder name, and balance.
- Encapsulate the data members using private access modifiers.
- Provide methods such as deposit(), withdraw(), transfer(), and displayAccountDetails().
- Declare an abstract method calculateInterest() in the Account class.
- Create a SavingsAccount class that extends Account and contains an interest rate.
- Override calculateInterest() in SavingsAccount.

- Create a CurrentAccount class that extends Account and contains an overdraft limit.
- Override calculateInterest() in CurrentAccount.
- Create objects of SavingsAccount and CurrentAccount in the main() method.
- Perform deposit, withdrawal, interest calculation, and transfer operations.
- Demonstrate polymorphism and dynamic binding by storing child-class objects in an Account reference and invoking the overridden calculateInterest() method.
- Display the final account details.

UML Class Diagram




+ calculateInterest() : double
SavingsAccount
CurrentAccount
- interestRate : double
- overdraftLimit: double
+ calculateInterest()
+ calculateInterest()


Account
- accountNumber : int
- balance : double
- accountType : String
+ deposit() : void
+ withdraw(amount : double) : boolean
+ transfer(toAccount : Account, amount : double) : boolean
+ calculateInterest() : double
+ displayDetails() : void
SavingsAccount
- interestRate : double
+ calculateInterest() : double
CurrentAccount
- overdraftLimit : double
+ withdraw(amount : double) : boolean

Code :

```
abstract class Account {
   // Encapsulation: data members are private
   private String accountNumber;
   private String holderName;
   protected double balance;

   public Account(String accountNumber, String holderName, double balance)
{
      this.accountNumber = accountNumber;
      this.holderName = holderName;
      this.balance = balance;
   }

   public void deposit(double amount) {
      if (amount > 0) {
         balance += amount;
         System.out.println("Deposited: Rs." + amount);
      } else {
         System.out.println("Invalid deposit amount.");
      }
   }
```

```
public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.println("Withdrawn: Rs." + amount);
    } else {
        System.out.println("Insufficient balance or invalid amount.");
    }
}

public void transfer(Account receiver, double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        receiver.balance += amount;
        System.out.println(
            "Transferred Rs." + amount + " to account "
            + receiver.accountNumber
        );
    } else {
        System.out.println("Transfer failed.");
    }
}
```

```
    // Abstract method: overridden by child classes
    public abstract double calculateInterest();

    public void displayDetails() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + holderName);
        System.out.println("Balance       : Rs." + balance);
    }
}

class SavingsAccount extends Account {
    private double interestRate;

    public SavingsAccount(
            String accountNumber,
            String holderName,
            double balance,
            double interestRate) {

        super(accountNumber, holderName, balance);
        this.interestRate = interestRate;
    }
```

```
    // Method overriding
    @Override
    public double calculateInterest() {
        return balance * interestRate / 100;
    }
}

class CurrentAccount extends Account {
    private double overdraftLimit;

    public CurrentAccount(
            String accountNumber,
            String holderName,
            double balance,
            double overdraftLimit) {

        super(accountNumber, holderName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    // Method overriding
    @Override
    public void withdraw(double amount) {
```

```
        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance -= amount;
            System.out.println("Withdrawn: Rs." + amount);
        } else {
            System.out.println("Withdrawal exceeds overdraft limit.");
        }
    }

    // Current accounts do not earn interest in this example
    @Override
    public double calculateInterest() {
        return 0;
    }
}

public class BankAccountDemo {
    public static void main(String[] args) {

        // Creating objects
        SavingsAccount savings =
                new SavingsAccount("SA101", "Rahul", 10000, 5.0);

        CurrentAccount current =
```

```
        new CurrentAccount("CA201", "Priya", 15000, 5000);

System.out.println("===== SAVINGS ACCOUNT =====");
savings.displayDetails();

savings.deposit(2000);
savings.withdraw(1500);

System.out.println(
        "Interest: Rs." + savings.calculateInterest()
);

System.out.println("\n===== CURRENT ACCOUNT =====");
current.displayDetails();

current.deposit(5000);
current.withdraw(18000);

System.out.println(
        "Interest: Rs." + current.calculateInterest()
);

System.out.println("\n===== FUND TRANSFER =====");
```

```
        savings.transfer(current, 2000);

        System.out.println("\n===== FINAL DETAILS =====");

        savings.displayDetails();
        System.out.println();

        current.displayDetails();

        // Polymorphism and dynamic binding
        System.out.println("\n===== POLYMORPHISM =====");

        Account account1 = savings;
        Account account2 = current;

        System.out.println(
            "Savings Interest: Rs." + account1.calculateInterest()
        );

        System.out.println(
            "Current Interest: Rs." + account2.calculateInterest()
        );
    }
```

}

Output :

===== SAVINGS ACCOUNT =====

Account Number : SA101

Account Holder : Rahul

Balance : Rs.10000.0

Deposited: Rs.2000.0

Withdrawn: Rs.1500.0

Interest: Rs.525.0

===== CURRENT ACCOUNT =====

Account Number : CA201

Account Holder : Priya

Balance : Rs.15000.0

Deposited: Rs.5000.0

Withdrawn: Rs.18000.0

Interest: Rs.0.0

===== FUND TRANSFER =====

Transferred Rs.2000.0 to account CA201

===== FINAL DETAILS =====

Account Number : SA101

Account Holder : Rahul

Balance : Rs.10500.0

Account Number : CA201

Account Holder : Priya

Balance : Rs.4000.0

===== POLYMORPHISM =====

Savings Interest: Rs.525.0

Current Interest: Rs.0.0